

GENIFY

LECTURE 05

24-July-2025

TOPICS COVERED:

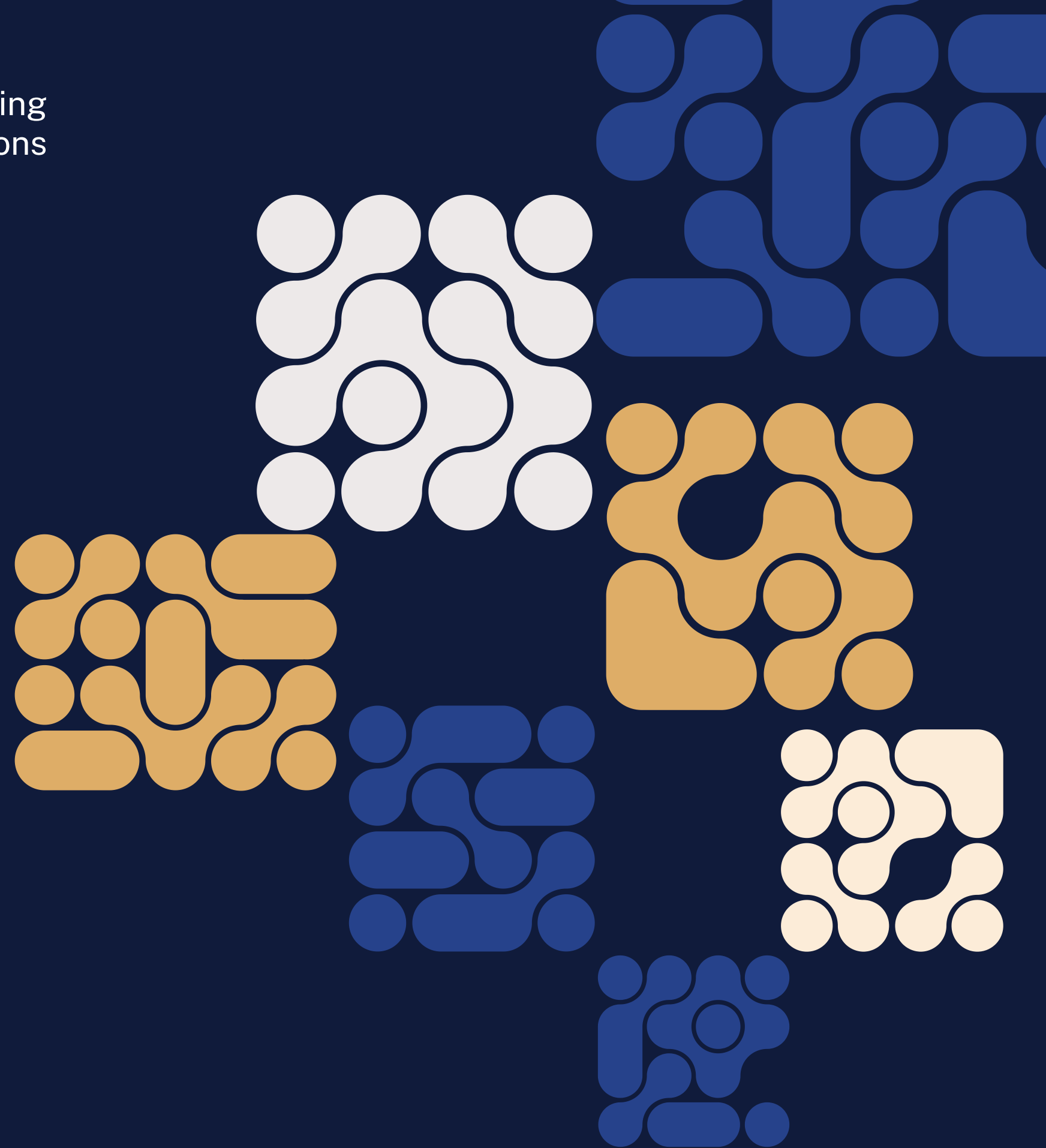
- MACHINE LEARNING
 - UNSUPERVISED ML
 - REINFORCEMENT ML

Presenter

Shahzaib Kashif

Co-Presenter

Shayan Hassan Baig



RECAP QUIZ TIME

Home Tasks

Discussion

T1: Discuss What do you conclude from the Evaluation Metrics of Logistic Regression.

T2: In K Means Clustering, Did we use the Y_{train} anywhere, and why (yes/no)

Logistic Regressoin Eval Metrics Explanation

1. **Accuracy:** Out of all the predictions I made, how many were correct?

2. **Precision:** When I say it's a YES, how often am I right?

3. **Recall:** How good am I at finding all the YES cases?

4. **F1-Score:** How good am I at both being right when I say YES and not missing YES cases? (Precision+Recall)

5. **AUC-ROC:** How good is my model at distinguishing YES vs NO across all thresholds? [0.5 (random) to 1.0 (perfect)]

6. **AUC-PR:** How well does my model maintain good precision and recall?

Metric	Value	What it means
Accuracy	0.805	80.5% of total predictions are correct
Precision	0.756	75.6% of predicted positives are truly positive
Recall	0.642	64.2% of actual positives were caught
F1 Score	0.694	Decent balance between precision and recall
AUC-ROC	0.857	Strong ability to separate classes
AUC-PR	0.720	Good performance focusing on positives

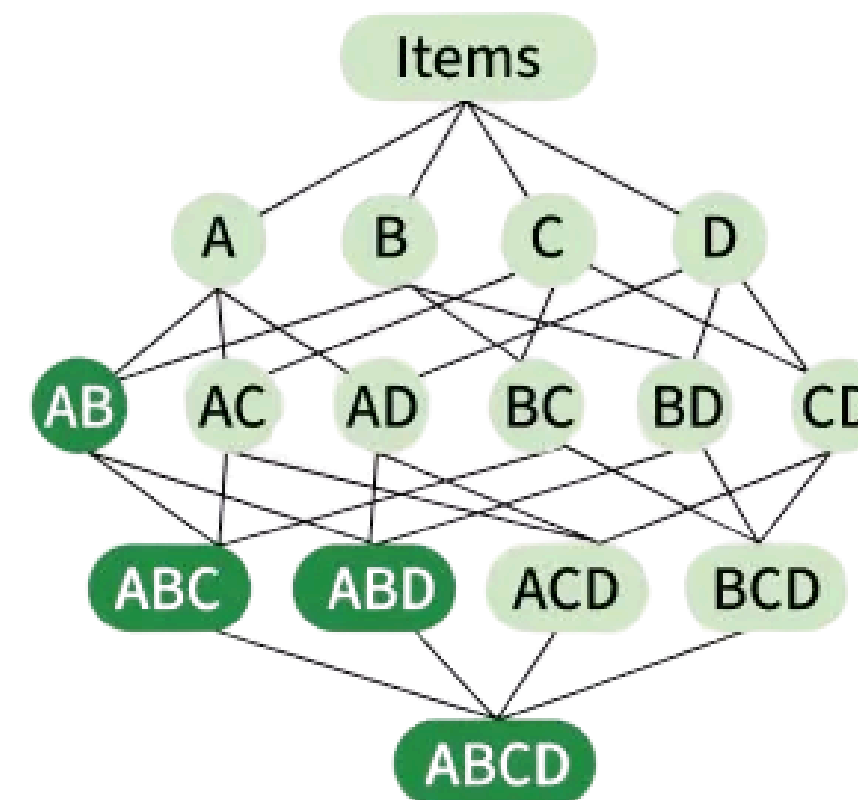


MACHINE LEARNING

Apriori Algorithm

- Apriori Algorithm is a basic method used in data analysis to find groups of items that often appear together in large sets of data.
- It helps to discover useful patterns or rules about how items are related which is particularly valuable in market basket analysis.
- Example: *Like in a grocery store if many customers buy bread and butter together, the store can use this information to place these items closer or create special offers. This helps the store sell more and make customers happy.*

The Apriori algorithm is a machine learning algorithm to identify relationships between items by identifying frequent itemsets.



Working of Apriori Algorithm

The Apriori Algorithm operates through a systematic process that involves several key steps:

- **1. Identifying Frequent Itemsets**

- The Apriori algorithm starts by looking through all the data to count how many times each single item appears. These single items are called 1-itemsets.
- Next it uses a rule called minimum support this is a number that tells us how often an item or group of items needs to appear to be important. If an item appears often enough meaning its count is above this minimum support it is called a frequent itemset.

- **2. Creating Possible item group**

- After finding the single items that appear often enough (frequent 1-item groups) the algorithm combines them to create pairs of items (2-item groups). Then it checks which pairs are frequent by seeing if they appear enough times in the data.
- This process keeps going step by step making groups of 3 items, then 4 items and so on. The algorithm stops when it can't find any bigger groups that happen often enough.

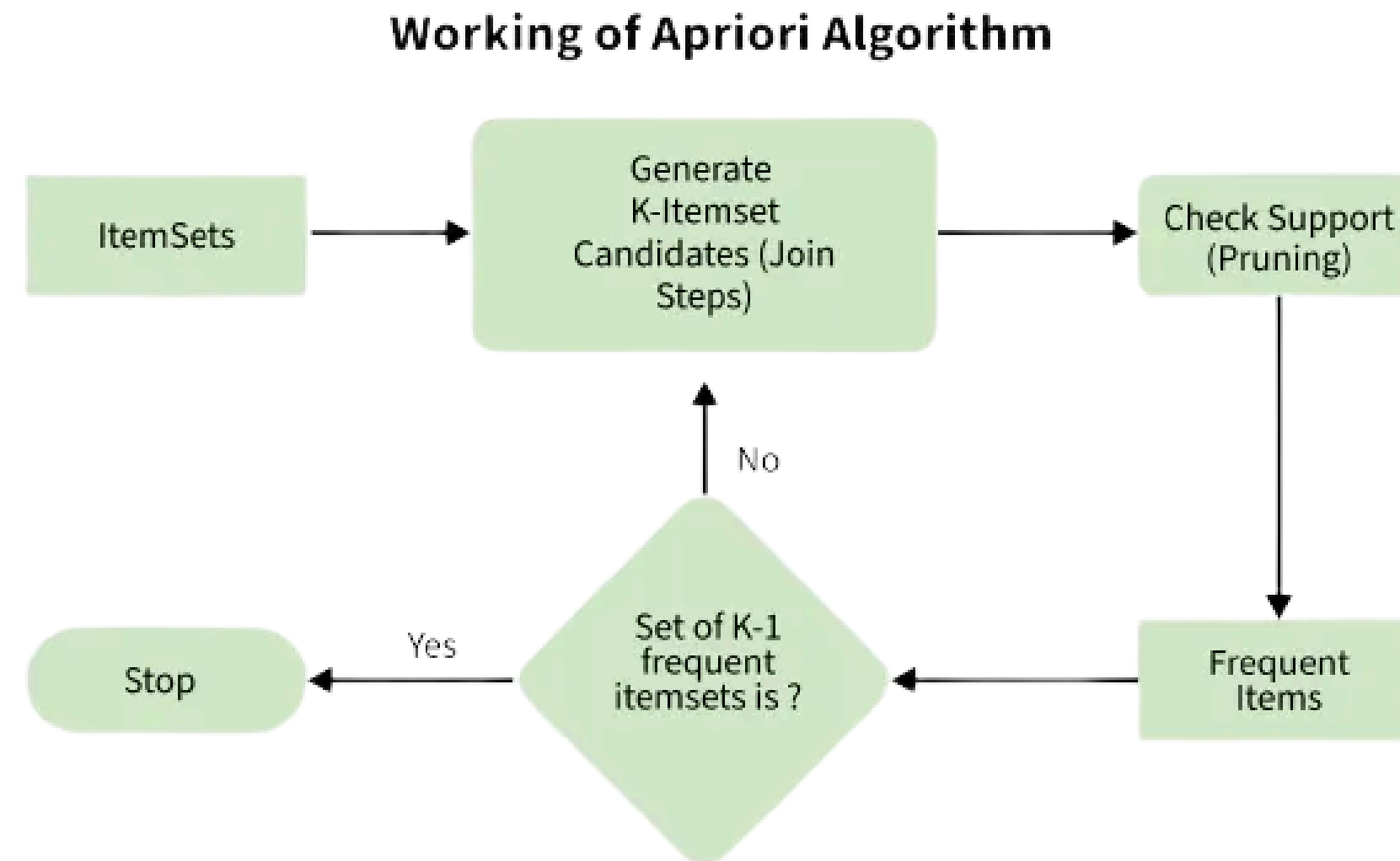
- **3. Removing Infrequent Item groups**

- The Apriori algorithm uses a helpful rule to save time. This rule says: if a group of items does not appear often enough then any larger group that includes these items will also not appear often.
- Because of this, the algorithm does not check those larger groups. This way it avoids wasting time looking at groups that won't be important making the whole process faster.

- **4. Generating Association Rules**

- The algorithm makes rules to show how items are related.
- It checks these rules using support, confidence and lift to find the strongest ones.

Working of Apriori Algorithm



Key Metrics of Apriori Algorithm

- **Support:** This metric measures how frequently an item appears in the dataset relative to the total number of transactions. A higher support indicates a more significant presence of the itemset in the dataset. Support tells us how often a particular item or combination of items appears in all the transactions ("Bread is bought in 20% of all transactions.")
- **Confidence:** Confidence assesses the likelihood that an item Y is purchased when item X is purchased. It provides insight into the strength of the association between two items. Confidence tells us how often items go together. ("If bread is bought, butter is bought 75% of the time.")
- **Lift:** Lift evaluates how much more likely two items are to be purchased together compared to being purchased independently. A lift greater than 1 suggests a strong positive association. Lift shows how strong the connection is between items. ("Bread and butter are much more likely to be bought together than by chance.")



Lift

Strength of itemset association

Confidence

Likelihood of itemset co-occurrence

Support

Frequency of itemset occurrence

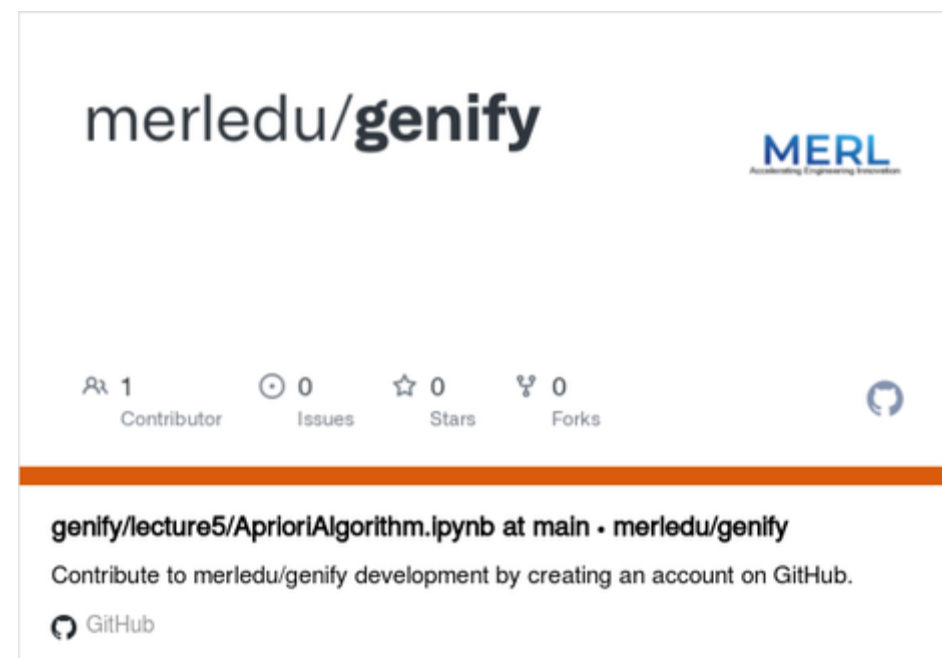
Applications of Apriori Algorithm

Below are some applications of Apriori algorithm used in today's companies and startups

1. **E-commerce:** Used to recommend products that are often bought together like laptop + laptop bag, increasing sales.
2. **Food Delivery Services:** Identifies popular combos such as burger + fries, to offer combo deals to customers.
3. **Streaming Services:** Recommends related movies or shows based on what users often watch together like action + superhero movies.
4. **Financial Services:** Analyzes spending habits to suggest personalised offers such as credit card deals based on frequent purchases.
5. **Travel & Hospitality:** Creates travel packages like flight + hotel by finding commonly purchased services together.
6. **Health & Fitness:** Suggests workout plans or supplements based on users' past activities like protein shakes + workouts.

Python Walkthrough

<https://github.com/merledu/genify/blob/main/lecture5/AprioriAlgorithm.ipynb>



Principal Component Analysis (PCA)

- PCA is a technique used in data analysis and machine learning. It helps you to reduce the number of features in a dataset while keeping the most important information.
- It changes your original features into new features these new features don't overlap with each other and the first few keep most of the important differences found in the original data.
- PCA is commonly used for data preprocessing for use with machine learning algorithms.
- It helps to remove redundancy, improve computational efficiency and make data easier to visualize and analyze especially when dealing with high-dimensional data.

Working of PCA

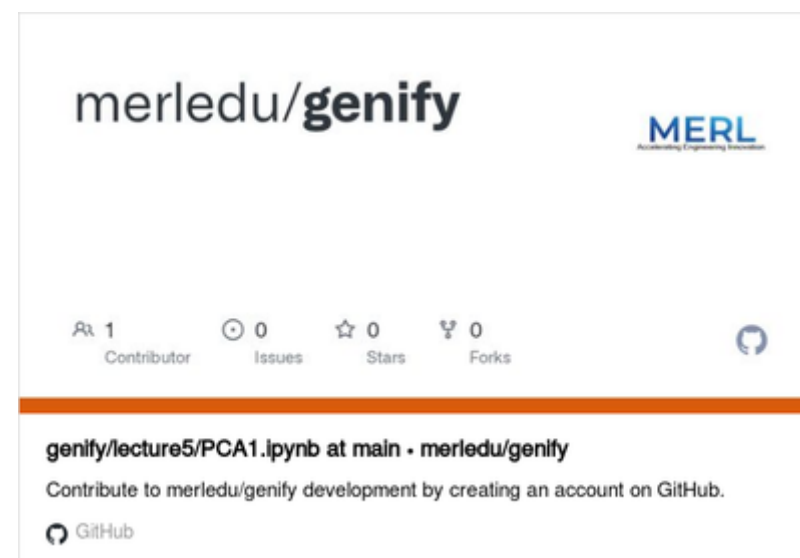
- PCA uses linear algebra to transform data into new features called principal components.
- It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix.
- PCA selects the top components with the highest eigenvalues and projects the data onto them to simplify the dataset.
- *Note: It prioritizes the directions where the data varies the most because more variation = more useful information.*
- Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns. Here's how it works step by step:
- Where to apply PCA?
 - Data Visualization
 - Speeding Up a Machine Learning (ML) Algorithm

What are Principal Components?

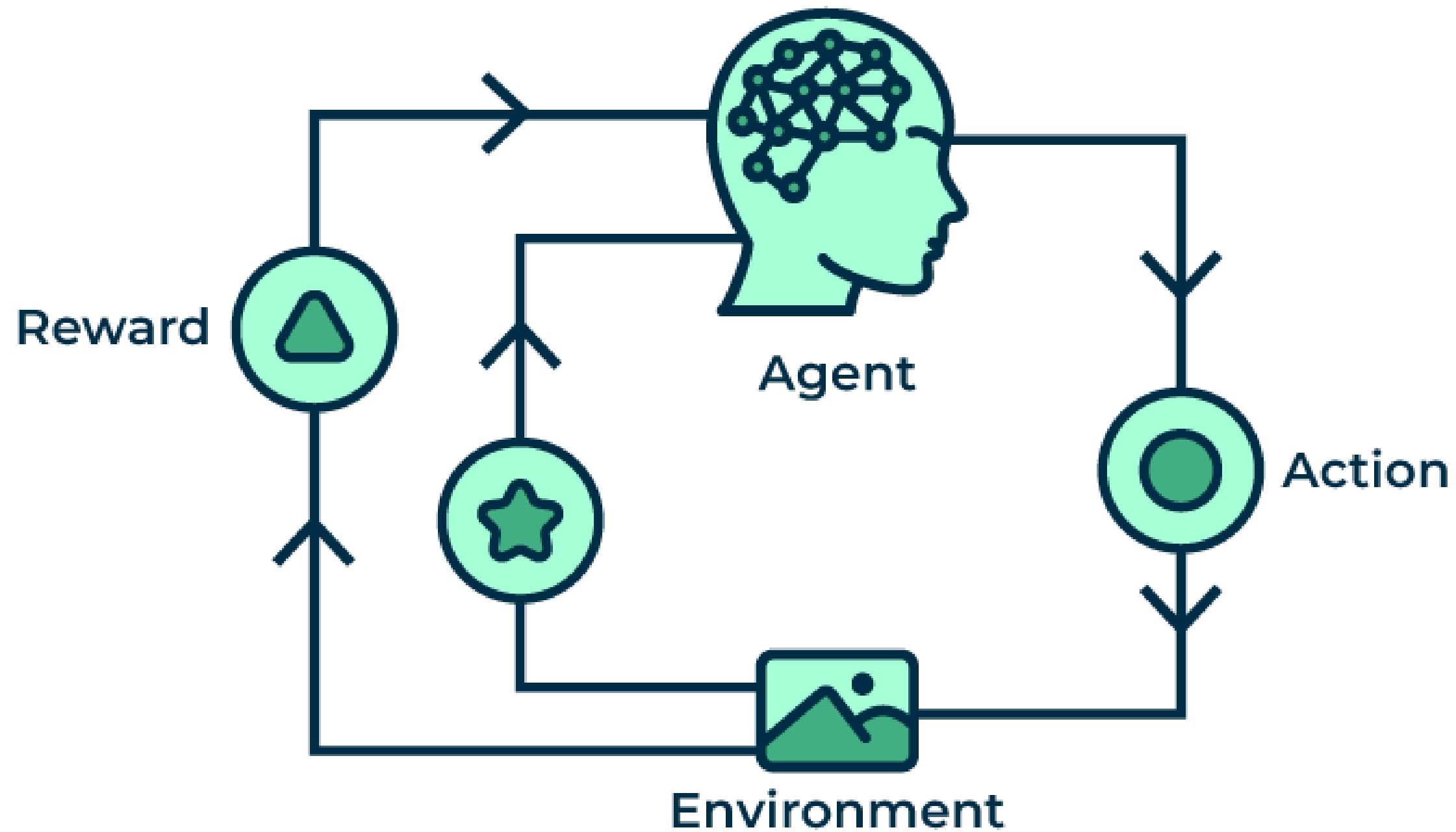
- They represent what's underneath the hood of your data
- In a layman term, when the data is projected into a lower dimension (assume three dimensions) from a higher space, the three dimensions are nothing but the three principal components that capture (or hold) most of the variance (information) of your data.
- Principal components have both direction and magnitude.
 - Direction tells you what features the PC is capturing and in what proportion.
 - Magnitude tells you how important that direction is for explaining the dataset.
- The principal components are a straight line, and the first principal component holds the most variance in the data.
- The reason you achieve uncorrelated principal components from the original features is that the correlated features contribute to the same principal component.
- Thereby reducing the original data features into uncorrelated principal components; each representing a different set of correlated features with different amounts of variability.
- Each principal component represents a percentage of the total variability captured from the data.

Python Walkthrough

<https://github.com/merledu/genify/blob/main/lecture5/PCA1.ipynb>

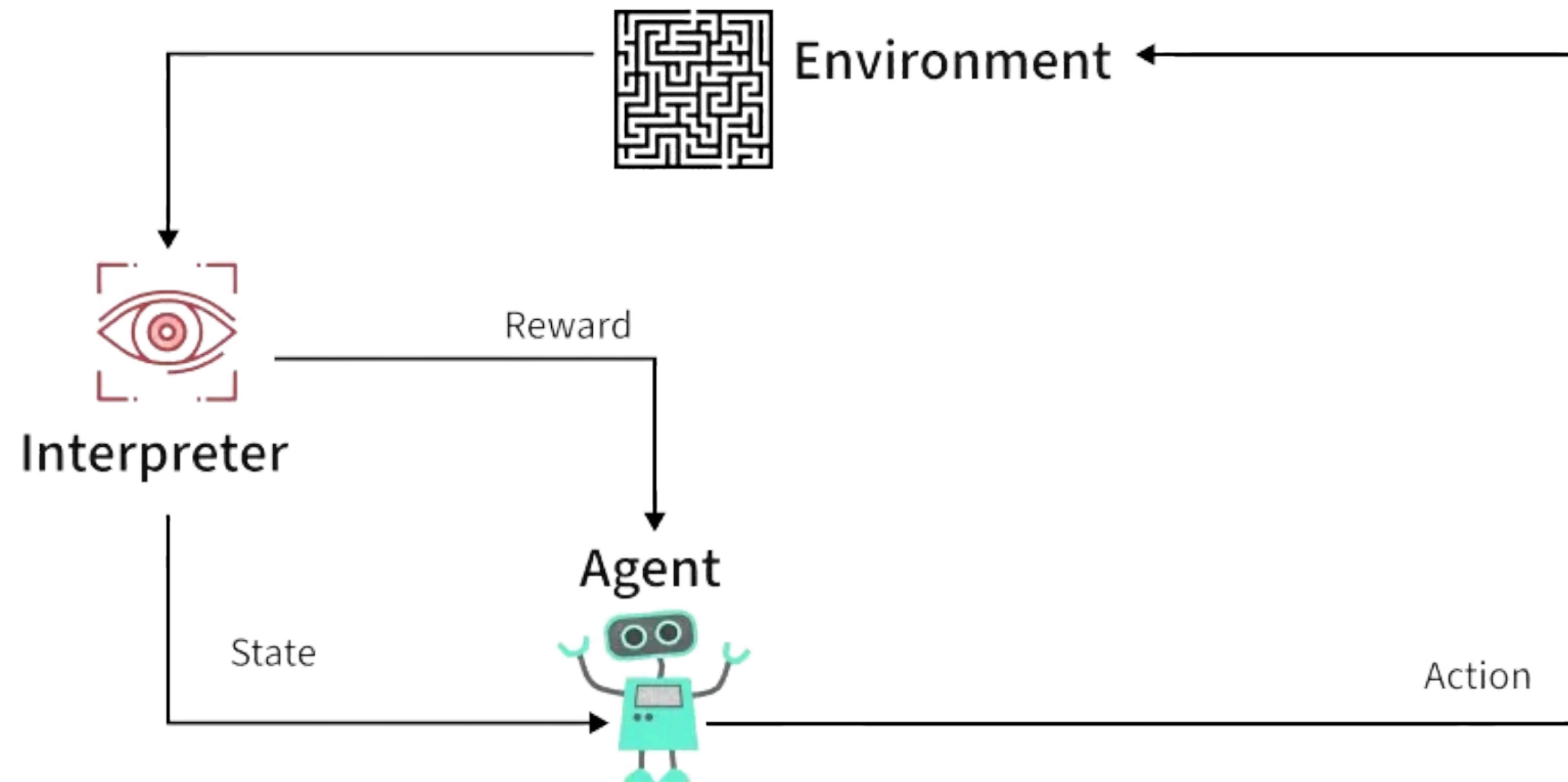


Reinforcement Learning



What is Reinforcement ML?

- Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards.
- RL allows machines to learn by interacting with an environment and receiving feedback based on their actions.
- This feedback comes in the form of rewards or penalties.
- Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal.
- The agent performs actions and receives feedback to optimize its decision-making over time.
 - Agent: The decision-maker that performs actions.
 - Environment: The world or system in which the agent operates.
 - Discrete and Continuous
 - State: The situation or condition the agent is currently in.
 - Action: The possible moves or decisions the agent can make.
 - Reward: The feedback or result from the environment based on the agent's action.



How Reinforcement Learning Works?

- The RL process involves an agent performing actions in an environment, receiving rewards or penalties based on those actions, and adjusting its behavior accordingly.
- This loop helps the agent improve its decision-making over time to maximize the cumulative reward.
- Here's a breakdown of RL components:
 - **Policy:** A strategy that the agent uses to determine the next action based on the current state.
 - **Reward Function:** A function that provides feedback on the actions taken, guiding the agent towards its goal.
 - **Value Function:** Estimates the future cumulative rewards the agent will receive from a given state.
 - **Model of the Environment:** A representation of the environment that predicts future states and rewards, aiding in planning.

Types of Reinforcement learning

1. Positive Reinforcement

- Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.
- **Advantages:** Maximizes performance, helps sustain change over time.
- **Disadvantages:** Overuse can lead to excess states that may reduce effectiveness.

2. Negative Reinforcement

- Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.
- **Advantages:** Increases behavior frequency, ensures a minimum performance standard.
- **Disadvantages:** It may only encourage just enough action to avoid penalties.

Reinforcement Learning Example: Navigating a Maze

- Imagine a robot navigating a maze to reach a diamond while avoiding fire hazards. The goal is to find the optimal path with the least number of hazards while maximizing the reward:
- Each time the robot moves correctly, it receives a reward.
- If the robot takes the wrong path, it loses points.
- The robot learns by exploring different paths in the maze. By trying various moves, it evaluates the rewards and penalties for each path. Over time, the robot determines the best route by selecting the actions that lead to the highest cumulative reward.

The robot's learning process can be summarized as follows:

- **Exploration:** The robot starts by exploring all possible paths in the maze, taking different actions at each step (e.g., move left, right, up, or down).
- **Feedback:** After each move, the robot receives feedback from the environment:
 - A positive reward for moving closer to the diamond.
 - A penalty for moving into a fire hazard.
- **Adjusting Behavior:** Based on this feedback, the robot adjusts its behavior to maximize the cumulative reward, favoring paths that avoid hazards and bring it closer to the diamond.
- **Optimal Path:** Eventually, the robot discovers the optimal path with the least number of hazards and the highest reward by selecting the right actions based on past experiences.



Application of Reinforcement Learning

- **Robotics:** RL is used to automate tasks in structured environments such as manufacturing, where robots learn to optimize movements and improve efficiency.
- **Game Playing:** Advanced RL algorithms have been used to develop strategies for complex games like chess, Go, and video games, outperforming human players in many instances.
- **Industrial Control:** RL helps in real-time adjustments and optimization of industrial operations, such as refining processes in the oil and gas industry.
- **Personalized Training Systems:** RL enables the customization of instructional content based on an individual's learning patterns, improving engagement and effectiveness.

Advantages of Reinforcement Learning

- **Solving Complex Problems:** RL is capable of solving highly complex problems that cannot be addressed by conventional techniques.
- **Error Correction:** The model continuously learns from its environment and can correct errors that occur during the training process.
- **Direct Interaction with the Environment:** RL agents learn from real-time interactions with their environment, allowing adaptive learning.
- **Handling Non-Deterministic Environments:** RL is effective in environments where outcomes are uncertain or change over time, making it highly useful for real-world applications.

Disadvantages of Reinforcement Learning

- **Not Suitable for Simple Problems:** RL is often an overkill for straightforward tasks where simpler algorithms would be more efficient.
- **High Computational Requirements:** Training RL models requires a significant amount of data and computational power, making it resource-intensive.
- **Dependency on Reward Function:** The effectiveness of RL depends heavily on the design of the reward function. Poorly designed rewards can lead to suboptimal or undesired behaviors.
- **Difficulty in Debugging and Interpretation:** Understanding why an RL agent makes certain decisions can be challenging, making debugging and troubleshooting complex.

1. Model-Based Methods

These methods use a model of the environment to predict outcomes and help the agent plan actions by simulating potential results.

- Markov decision processes (MDP)
- Bellman equation
- Value iteration algorithm
- Monte Carlo Tree Search

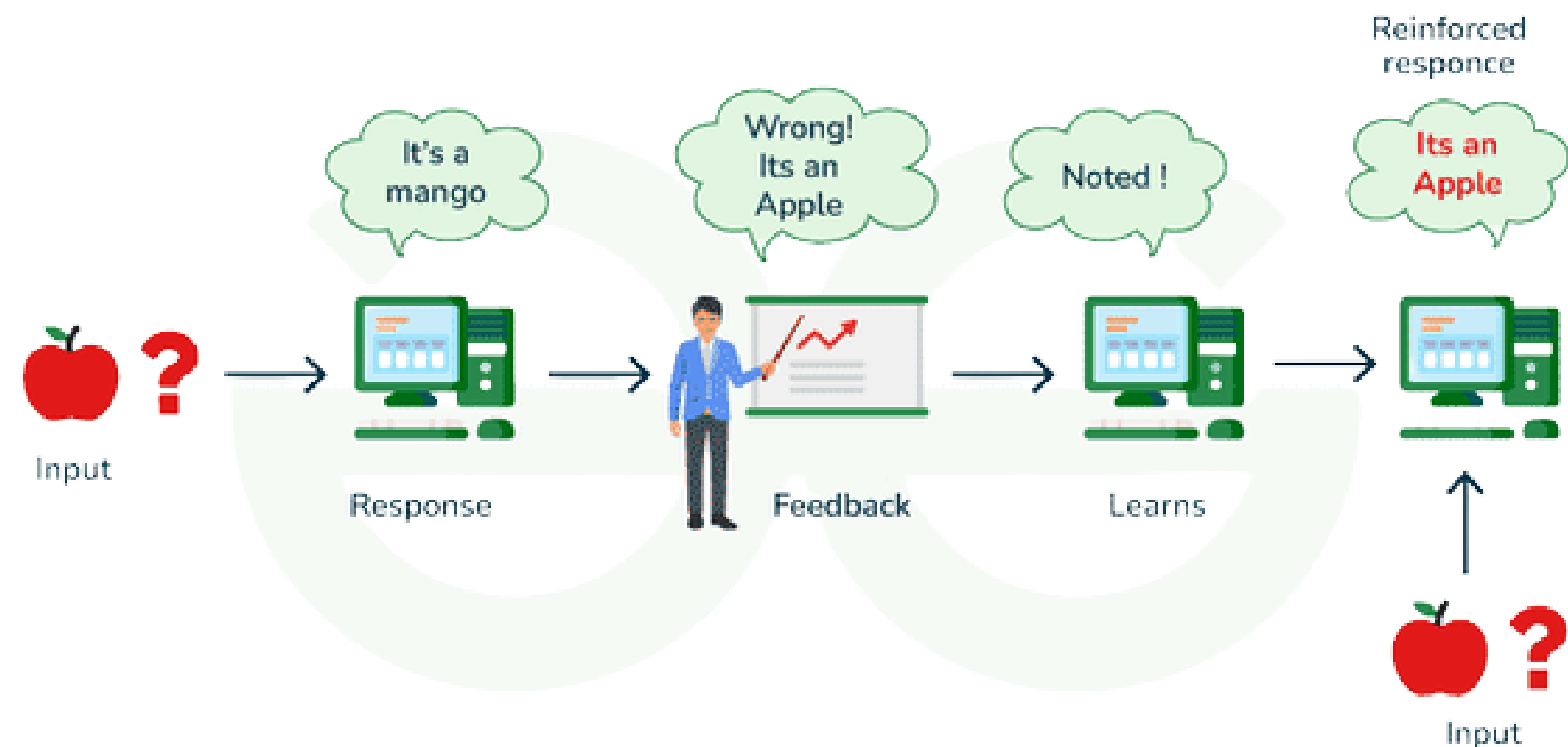
2. Model-Free Methods

The agent learns directly from experience by interacting with the environment and adjusting its actions based on feedback.

- Q-learning
- SARSA
- Monte Carlo Methods
- Reinforce algorithms
- Actor critic algorithm
- Asynchronous advantage actor-critic (A3C)

Q Learning

- Helps an agent learn how to make the best decisions by interacting with its environment. Instead of needing a model of the environment the agent learns purely from experience by trying different actions and seeing their results.
- The core idea is that the agent builds a Q-table which stores Q-values. Each Q-value estimates how good it is to take a specific action in a given state in terms of the expected future rewards. Over time the agent updates this table using the feedback it receives



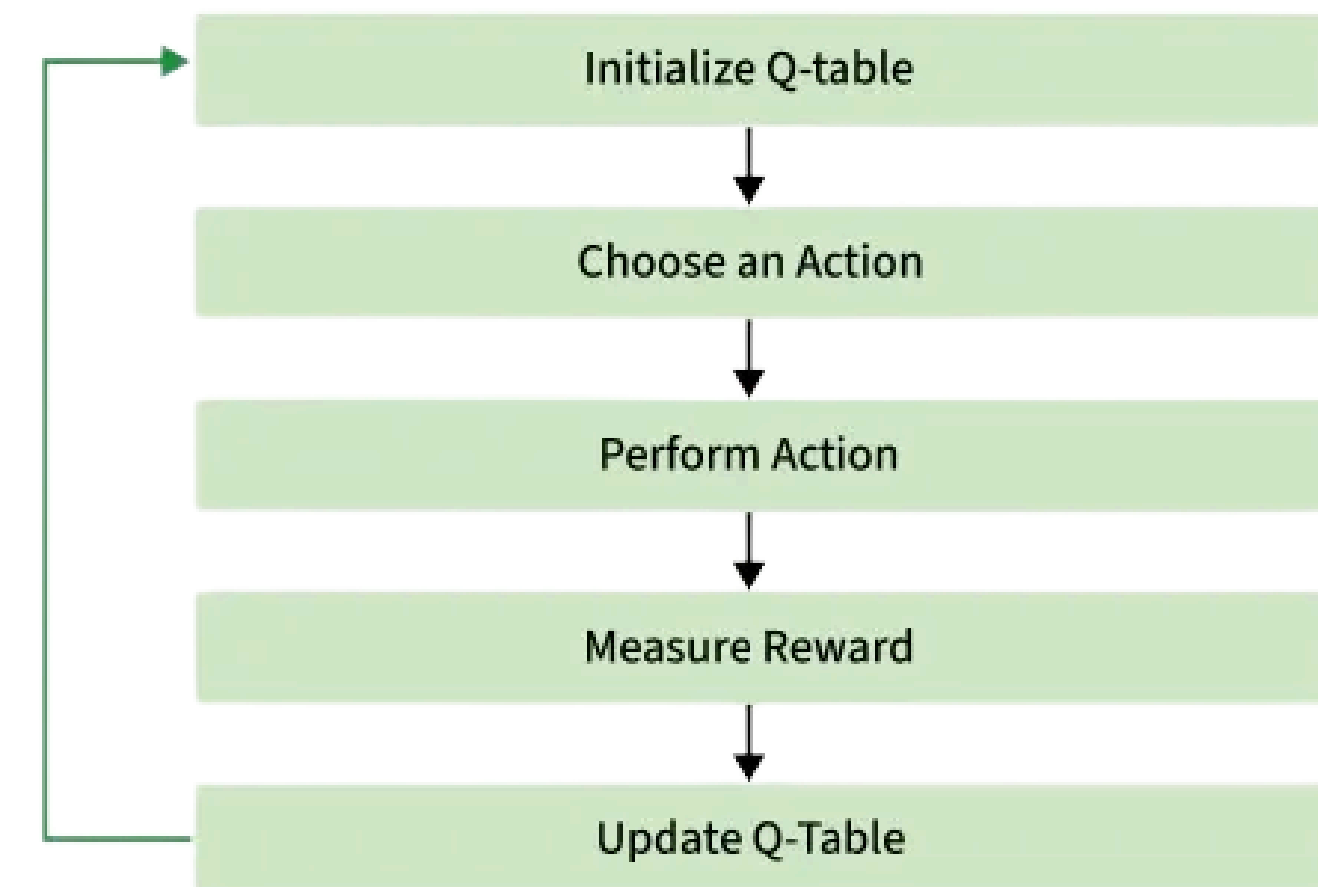
Working of Q Learning

Q-learning models follow an iterative process where different components work together to train the agent. Here's how it works step-by-step.

1. Start at a State (S)
2. Agent Selects an Action (A)
3. Action is Executed and Environment Responds
4. Learning Algorithm Updates the Q-Table
5. Policy is Refined and the Cycle Repeats

Multiple iterations till
the model achieves
termination state

Q learning algorithm



Python Walkthrough

<https://github.com/merledu/genify/blob/main/lecture5/ReinforcementLearning.ipynb>

